

DOCUMENTATION COMPLÈTE - ALTERDUNE

Projet de Programmation Orientée Objet en C++

Auteurs : Amandine Baranger & Thibault Sevin Allouet

Langage : C++

-- TABLE DES MATIÈRES -----

1. [Vue d'ensemble](#)
 2. [Architecture & Hiérarchie des Classes](#)
 3. [Description des Objets](#)
 4. [Méthodes & Fonctions](#)
 5. [Règles du Jeu](#)
 6. [Système de Combat](#)
 7. [Chargement des Données](#)
-

-- VUE D'ENSEMBLE -----

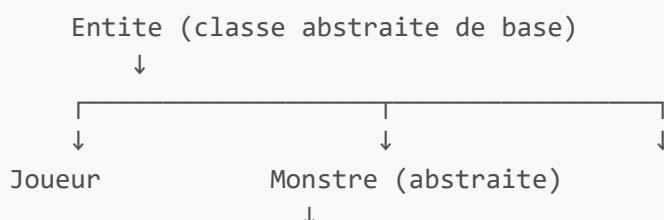
ALTERDUNE est un jeu RPG au tour par tour en ligne de commande. Le joueur affronte des monstres de trois catégories différentes dans des combats stratégiques où il doit atteindre **10 victoires** pour terminer la partie.

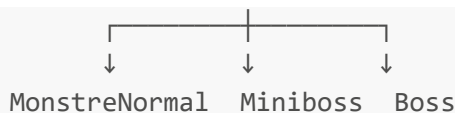
Caractéristiques principales :

- ☒ **Système Mercy** : Possibilité d'épargner les monstres au lieu de les tuer
 - ☒ **Actions ACT** : 10 actions pré-définies modifiant la jauge Mercy
 - ☒ **Inventaire** : Gestion d'items consommables (HEAL)
 - ☒ **Bestiaire** : Enregistrement des monstres vaincus (tués/épargnés)
 - ☒ **Fins multiples** : Génocidaire / Pacifiste / Neutre
 - ☒ **Équilibre de jeu** : Dégâts entre 0-70% du HP max
-

-- ARCHITECTURE & HIÉRARCHIE DES CLASSES -----

Diagramme d'héritage :





Relations de composition :

```

Jeu
├── Joueur (1)
│   ├── vector<Item> (inventaire)
│   └── Bestiaire (monBestiaire)
│       └── vector<EntreeBestiaire>
├── vector<Monstre*> (baseMonstres)
└── map<string, ActionAct> (catalogueActions)
  
```

-- DESCRIPTION DES OBJETS -----

ENTITE (Classe de base)

```

class Entite {
protected:
    string nom;           // Nom du personnage/monstre
    int hpActuel;         // Points de vie actuels
    int hpMax;            // Points de vie maximums

public:
    string getNom();
    int getHpActuel();
    int getHpMax();
    void recevoirDegats(int degats);
    bool estVivant();
    void restaurerHp();
};
  
```

Responsabilités :

- Gère les HP du joueur/monstre
- Détermine si l'entité est vivante

JOUEUR (Héritage d'Entite)

```

class Joueur : public Entite {
private:
    int monstresTues;      // Nombre de monstres tués
  
```

```

int monstresEpargnes;    // Nombre de monstres épargnés
int victoires;          // Total victoires (0-10)
vector<Item> inventaire; // Items possédés (COMPOSITION)
Bestiaire monBestiaire; // Historique combats (COMPOSITION)

```

Méthodes principales :

Méthode	Paramètres	Description
attaquer()	Monstre& cible	Inflige dégâts aléatoires (30-70% HP max)
agirACT()	Monstre& cible, ActionAct action	Exécute une action ACT modifiant Mercy
epargnerMercy()	Monstre& cible	Épargne le monstre si Mercy $\geq$ 100
ajouterItem()	Item objet	Ajoute un item à l'inventaire
afficherInventaire()	-	Affiche tous les items (nom, HP, quantité)
utiliserItem()	int index	Consomme un item et restaure HP

Accesseurs :

```

int getVictoires();
int getMonstresTues();
int getMonstresEpargnes();
vector<Item>& getInventaire();
Bestiaire& getBestiaire();

```

MONSTRE (Classe abstraite)

```

class Monstre : public Entite {
protected:
    int attaque;          // Puissance d'attaque
    int defense;          // Défense (non utilisée)
    int mercyGoal;        // Objectif Mercy (100)
    int mercyActuel;      // Mercy actuelle (0-100)
    string actionsPossibles[4]; // 4 actions max : normal=2, miniboss=3,
    boss=4

```

Méthodes :

Méthode	Description
attaquer()	Fait 30-70% dégâts au joueur

Méthode	Description
<code>modifierMercy()</code>	Modifie Mercy (bornée 0-100)
<code>peutEtreEpargne()</code>	Retourne true si Mercy $\geq$ 100
<code>getCategory()</code>	Virtuelle pure - retourne "NORMAL"/"MINIBOSS"/"BOSS"
<code>getAction()</code>	Retourne l'action à l'index i

MONSTRENORMAL, MINIBOSS, BOSS (Dérivées concrètes)

```
class MonstreNormal : public Monstre {
public:
    string getCategory() override { return "NORMAL"; } // 2 actions
};

class Miniboss : public Monstre {
public:
    string getCategory() override { return "MINIBOSS"; } // 3 actions
};

class Boss : public Monstre {
public:
    string getCategory() override { return "BOSS"; } // 4 actions
};
```

Polymorphisme : Chaque catégorie retourne sa catégorie et le nombre d'actions disponibles.

ACTIONACT (Système ACT)

```
class ActionAct {
private:
    string id; // COMPLIMENT, JOKE, INSULT, etc.
    string texte; // Texte affiché quand exécutée
    int impactMercy; // Modification Mercy (+20 ou -25)

public:
    string getId();
    int executer(); // Affiche le texte et retourne impact
};
```

10 Actions disponibles :

Action	Impact	Type
COMPLIMENT	+20	Positive

Action	Impact	Type
JOKE	+15	Positive
OBSERVE	+15	Positive
PET	+25	Positive
OFFER_SNACK	+30	Positive
DISCUSS	+20	Positive
REASON	+18	Positive
DANCE	+10	Positive
INSULT	-20	Négative
TAUNT	-25	Négative

ITEM

```
class Item {
private:
    string nom;           // Nom de l'item
    string type;          // Type (HEAL)
    int valeur;           // HP restaurés
    int quantite;         // Quantité disponible

public:
    string getNom();
    int getQuantite();
    int getValeur();      // Retourne les HP que l'item soigne
    int utiliser();       // Retourne la valeur
    void reduireQuantite(); // -1 à la quantité
};
```

BESTIAIRE & ENTREEBESTIAIRE

```
class Bestiaire {
private:
    vector<EntreeBestiaire> entrees; // Historique des monstres vaincus

public:
    void ajouterEntree(Monstre& m, string resultat); // "Tue" ou "Epargne"
    void afficher(); // Liste tous les monstres vaincus
};

class EntreeBestiaire {
private:
```

```
    string nom, categorie, resultat;
    int hpMax;
};
```

JEU (Classe principale)

```
class Jeu {
private:
    map<string, ActionAct> catalogueActions; // Les 10 actions pré-définies
    vector<Monstre*> baseMonstres;           // Tous les monstres chargés du CSV

public:
    Jeu(); // Initialise le catalogue
    ~Jeu(); // Libère la mémoire
    void demarrer(); // Boucle principale
    void chargerFichiers(); // Charge CSV
    void boucleCombat(); // Gère le combat tour par tour
    void verifierFinDePartie(); // Détermine la fin
    (génocidaire/pacifiste/neutre)
};
```

-- MÉTHODES & FONCTIONS -----

Jeu::demarrer() - Boucle principale

1. Charge les fichiers (monstres.csv, items.csv)
2. Demande le nom du joueur
3. Crée l'instance Joueur avec 50 HP
4. MENU PRINCIPAL (boucle) :
 - Option 1 : Bestiaire
 - Option 2 : Démarrer un combat
 - Option 3 : Statistiques
 - Option 4 : Inventaire
 - Option 5 : Quitter
5. Vérifie fin de partie si 10 victoires atteintes

Jeu::chargerFichiers() - Chargement CSV

monstres.csv :

```

categorie;nom;hp;atk;def;mercyGoal;act1;act2;act3;act4
NORMAL;Sandprowler;30;7;1;100;COMPLIMENT;JOKE;-;-
BOSS;Baron Harkonnen;90;20;6;100;COMPLIMENT;TAUNT;OBSERVE;REASON

```

items.csv :

```

nom;type;valeur;quantite
Gourde de Qat;HEAL;15;3

```

Jeu::boucleCombat() - Tour par tour

```

TANT QUE (joueur vivant ET monstre vivant) :
    AFFICHE : Menu joueur (FIGHT/ACT/ITEM/MERCY)

    SI FIGHT :
        Calcul dégâts aléatoires (30-70% HP monster)
        Monstre recoit dégâts
        SI monstre mort → Bestiaire.ajouterEntree("Tue")

    SI ACT :
        Affiche actions disponibles du monstre
        Joueur choisit action
        Action modifie Mercy du monstre

    SI ITEM :
        Affiche inventaire
        Joueur choisit item
        HP restaurés (item consommé)

    SI MERCY :
        SI Mercy ≥ 100 → Monstre épargné
        → Bestiaire.ajouterEntree("Epargne")

    SI Monstre vivant :
        Monstre attaque joueur
        SI joueur mort → Combat perdu

```

-- RÈGLES DU JEU -----

Démarrage

- Le joueur nomme son personnage
- Démarre avec **50 HP max**
- Charge **20 items** et **25 monstres** du CSV

Objectif

- Remporter **10 victoires** (Tués + Épargnés)

Combat - Système de Dégâts

Formule équilibrée :

```
int pourcentage = 30 + rand() % 41;          // Entre 30% et 70%
int maxDamage = (hpMax * pourcentage) / 100;
int degats = rand() % (maxDamage + 1);      // Entre 0 et maxDamage
```

Avantages :

- ☒ Combat moins frustrant (dégâts max = 70% HP)
- ☒ Stratégie > chance
- ☒ Items importants (peuvent sauver)

Système Mercy

Condition	Action	Résultat
Mercy < 100	Impossible épargner	Combat continue
Mercy = 100	MERCY disponible	Monstre épargné, victoire
Action positive	+15 à +30 Mercy	Monstre s'apaise
Action négative	-20 à -25 Mercy	Monstre s'énerve

Bornage : Mercy ∈ [0, 100]

Inventaire

- Items **HEAL** uniquement
- Chacun soigne X HP
- Consommable une seule fois
- Utilisable hors-combat (menu) ou pendant le combat

Fin de Partie

Après **10 victoires** :

Type	Condition	Message
GÉNOCIDAIRE	0 monstre épargné	"Tous ont été tués"
PACIFISTE	0 monstre tué	"Tous ont été épargnés"
NEUTRE	Mix tués/épargnés	"Approche mixte"

-- SYSTÈME DE COMBAT -----

Menu combat joueur à chaque tour

```
Vos HP: 35/50 | Monstre HP: 14/20
1. FIGHT (Attaquer)
2. ACT (Agir)
3. ITEM (Objets)
4. MERCY (Epargner)
Choix :
```

Actions disponibles selon catégorie

Catégorie	Nb Actions	Exemples
NORMAL	2	COMPLIMENT, JOKE
MINIBOSS	3	OBSERVE, PET, OFFER_SNACK
BOSS	4	REASON, DANCE, JOKE, INSULT

Exemple tour complet

```
--- TOUR DE Toto ---
Vos HP: 50/50 | Monstre HP: 20/20
1. FIGHT (Attaquer)
2. ACT (Agir)
3. ITEM (Objets)
4. MERCY (Epargner)
Choix : 1

Toto lance une attaque sur Dunemite !
Coup réussi ! Dunemite perd 8 HP.

--- TOUR DU MONSTRE ---
Dunemite vous attaque !
Vous subissez 12 points de degats.
```

-- CHARGEMENT DES DONNÉES -----

Fichiers obligatoires

monstres.csv (25 monstres)

```

categorie;nom;hp;atk;def;mercyGoal;act1;act2;act3;act4

NORMAL;Sandprowler;30;7;1;100;COMPLIMENT;JOKE;-;-
NORMAL;Dunemite;20;6;1;100;OFFER_SNACK;DISCUSS;-;-
MINIBOSS;Stillsuit Phantom;45;12;3;100;OBSERVE;PET;OFFER_SNACK;-
BOSS;Shai-Hulud Jr;80;18;5;100;REASON;DANCE;JOKE;INSULT

```

items.csv (20 items)

```

nom;type;valeur;quantite

Gourde de Qat;HEAL;15;3
Barre Protéique Fremen;HEAL;8;5
Elixir de Ver;HEAL;50;1

```

Gestion d'erreurs

```

if (!fichierMonstres.is_open()) {
    cerr << "Erreur : fichier monstres.csv introuvable." << endl;
    exit(1);
}

try {
    int hp = stoi(hpStr);
    // ...
} catch (...) {
    continue; // Ignorer ligne mal formée
}

```

-- RÉSUMÉ - CONCEPTS POO -----

- ✓ **Encapsulation** : Attributs privés, accesseurs publics
- ✓ **Héritage** : Entite → Joueur/Monstre ; Monstre → 3 catégories
- ✓ **Polymorphisme** : `getCategory()` override, relations virtuelles
- ✓ **Composition** : Joueur contient Inventaire + Bestiaire
- ✓ **Abstraction** : Classes abstraites (Entite, Monstre)

-- COMPILATION & EXÉCUTION -----

```
g++ -std=c++17 -o Alterdune.exe \  
    main.cpp actionAct.cpp bestiaire.cpp boss.cpp \  
    entite.cpp entreebestiaire.cpp item.cpp jeu.cpp \  
    joueur.cpp miniboss.cpp monstre.cpp monstrenormal.cpp -I.  
  
./Alterdune.exe
```

-- NOTES FINALES -----

Ce projet met en pratique les 4 piliers de la POO :

1. **Entite** = classe de base commune
2. **Joueur/Monstre** = héritage et comportements différents
3. **Polymorphisme** = actions ACT adaptées par catégorie
4. **Composition** = Inventaire et Bestiaire intégrés

Le système Mercy et les 10 actions différentes font du jeu une **expérience stratégique** où le joueur doit choisir ses tactiques (combat direct ou diplomatie).

FIN DE DOCUMENTATION

Généré : Mai 2026